

Resumen de los temas importantes

Una **tabla hash** es una estructura de datos que asocia claves con valores, permitiendo un acceso rápido a los datos. La tabla hash se implementa utilizando un arreglo y una **función hash**, que convierte una clave en un índice dentro de ese arreglo.

- **Resolución de colisiones:** Una **colisión** ocurre cuando la función hash produce el mismo índice para dos claves diferentes. Para manejar esto, la implementación proporcionada utiliza **encadenamiento**. Esto significa que cada celda del arreglo es en realidad una lista (en este caso, una `List<T>`). Si se produce una colisión, el nuevo elemento simplemente se añade al final de la lista en ese índice.
- **Hashing universal:** La implementación usa **hashing universal**. Esto garantiza que, en promedio, el número de colisiones será bajo, independientemente de la distribución de las claves de entrada. La función `universalHash(long k)` en el código implementa esta técnica con la fórmula $((a * k + b) \bmod p) \bmod \text{size}$, donde **p** debe ser un número primo grande y **a** y **b** pueden ser seleccionados aleatoriamente.
- **Implementación genérica:** Para que la tabla hash pueda funcionar con cualquier tipo de objeto, se utiliza la **programación genérica de Java** con la sintaxis

`<T extends HashableKey>`

Esto indica que la tabla puede almacenar objetos de cualquier clase T, siempre y cuando esa clase implemente la interfaz `HashableKey`.

- **Interfaz `HashableKey`:** Esta interfaz obliga a cualquier clase que la implemente a tener un método llamado `getHashKey()`. Este método debe devolver un `String` que se utilizará como la clave para calcular el valor hash del objeto.
- **Ejemplo con la clase `Persona`:** La clase `Persona` implementa `HashableKey` y su método `getHashKey()` devuelve una combinación del primer nombre y el primer apellido, lo que se convierte en la clave para el hashing.
- **Manejo de claves y `equals()`:** La función `stringToInt(String s)` convierte la clave de tipo `String` en un valor numérico (`long`) para ser procesado por la función universal hash. Es crucial que las clases que se almacenen en la tabla implementen correctamente los métodos `equals()` y `hashCode()` para que las operaciones de `search()`, `insert()` y `remove()` funcionen como se espera. En el ejemplo, el método `equals` de la clase `Persona` compara los objetos basándose en el valor de `getHashKey()`.

Pseudocódigo para la implementación de la tabla Hash

INTERFAZ HashableKey

 FUNCION getHashKey() retorna String

FIN INTERFAZ

CLASE UniversalHashTable<T> donde T implementa HashableKey

 // Atributos

 arreglo de Listas<T> tabla

 entero tamaño

 long a, b, p

 // Constructor 1: Inicializa con tamaño

 CONSTRUCTOR UniversalHashTable(entero tamaño)

 establecer this.tamaño = tamaño

 crear arreglo de Listas<T> de tamaño 'tamaño' y asignarlo a 'tabla'

 PARA i DESDE 0 HASTA 'tamaño' - 1

 crear una nueva lista vacía en 'tabla[i]'

 FIN PARA

 p = 2147483647 // Un número primo grande

 a = 1 + entero aleatorio entre 1 y p-1

 b = entero aleatorio entre 0 y p-1

 // Constructor 2: Inicializa con parámetros específicos

 CONSTRUCTOR UniversalHashTable(entero tamaño, long a, long b, long p)

 establecer this.tamaño = tamaño

 crear arreglo de Listas<T> de tamaño 'tamaño' y asignarlo a 'tabla'

 PARA i DESDE 0 HASTA 'tamaño' - 1

 crear una nueva lista vacía en 'tabla[i]'

 FIN PARA

 establecer this.p = p

 establecer this.a = a

 establecer this.b = b

 // Método universalHash

 FUNCION universalHash(long k) retorna entero

 long clave = ((this.a * k + this.b) % this.p)

 retornar (entero)(clave % this.tamaño)

```
// Método stringToInt
FUNCION stringToInt(String s) retorna long
    long hash = 0
    entero base = 31
    PARA i DESDE 0 HASTA longitud de s - 1
        hash = (hash * base + caracter en s en posición i) % p
    FIN PARA
    retornar hash
```

```
// Método insert
FUNCION insert(objeto T obj)
    String claveString = obj.getKey()
    long claveInt = stringToInt(claveString)
    entero indice = universalHash(claveInt)

    SI tabla[indice] NO contiene obj
        tabla[indice].agregar(obj)
    FIN SI
```

```
// Método search
FUNCION search(String clave) retorna T
    long claveInt = stringToInt(clave)
    entero indice = universalHash(claveInt)

    POR CADA elemento EN tabla[indice]
        SI elemento.equals(clave)
            retornar elemento
        FIN SI
    FIN POR CADA
```

```
retornar null // No se encontró
```

```
// Método remove
FUNCION remove(objeto T obj) retorna booleano
    String claveString = obj.getKey()
    long claveInt = stringToInt(claveString)
    entero indice = universalHash(claveInt)

    retornar tabla[indice].remove(obj)
```

```

// Método printTable
FUNCION printTable()
    IMPRIMIR "Parámetros: a: " + a + ", b: " + b + ", p: " + p
    PARA i DESDE 0 HASTA `tamaño` - 1
        IMPRIMIR i + " -> " + `tabla[i]`
    FIN PARA

FIN CLASE

```

```

CLASE Persona implementa HashableKey
// Atributos
String id, primerNombre, segundoNombre, primerApellido, segundoApellido, fechaNacimiento

// Constructor
CONSTRUCTOR Persona(String id, ...)    // Inicializar atributos

// Implementación del método de la interfaz
@Override
FUNCION getHashKey() retorna String
    retornar primerNombre + primerApellido

// Métodos 'toString' y 'equals'
@Override
FUNCION toString() retorna String
    Retornar id + ": " + primerNombre + segundoNombre + ...

@Override
FUNCION equals(Object o) retorna booleano
    SI o es null retornar false

    SI o es una instancia de String
        retornar this.getHashKey().equals(o)

    SI o es la misma instancia (this == o) retornar true

    SI o NO es una instancia de Persona retornar false

    Persona persona = (Persona) o
    retornar this.getHashKey().equals(persona.getHashKey())

FIN CLASE

```